

Terraform Main Concepts

Providers, resources, dependencies and the all-important Terraform state — the vocabulary you'll use every day.

CLI & Providers

Resources

Dependencies

State & lock file

Lab 02

Today's roadmap



1 - CLI & Providers

The command line, providers & provisioners, and version constraints.



2 - Resources

The building blocks, how to reference them, and dependency mapping.



3 - State & the lock file

What state is, why it's critical, and how the plan is computed.



4 - Lab 02

Organize your repo project into main / variables / outputs / versions.

01

The CLI & Providers

One binary, and the plugins that let it talk to every platform.

The Terraform CLI

A single Go binary that takes your `.tf` files as input — built to run both by hand and inside automation.

- Cross-platform: Windows, Linux, macOS, ARM
- The real value is **unattended** runs in pipelines

init

Prepare the working dir

fmt

Auto-format code

validate

Check syntax

plan

Preview changes

apply

Make it real

destroy

Tear it down

Terraform Providers

Terraform embraces each platform through a **provider** — one HCL language, many targets. Anything with a solid API or CLI can have one.



Cloud

Azure · AWS · GCP



DevOps platforms

GitHub · GitLab · Azure DevOps



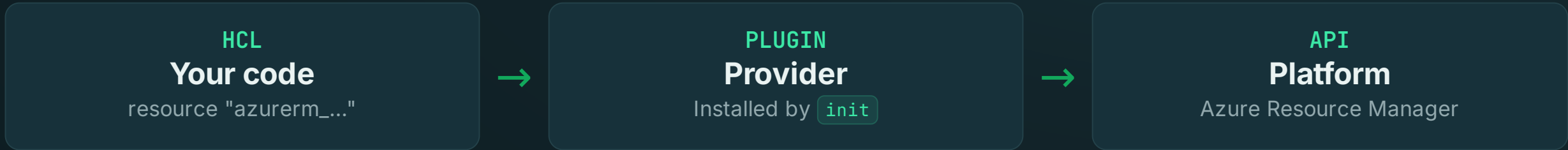
SaaS & more

New Relic · Twingate · Tailscale



Code for Azure won't build in AWS — the provider abstracts **the language**, not the resources. Need custom? Write your own in Go.

How a provider talks to the platform



↓ Providers are downloaded by `terraform init` — always the first command you run in a new working directory.

Declaring providers & versions

HCL

```
terraform {  
  required_version = ">= 1.9"  
  required_providers {  
    azurerm = {  
      source  = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}  
  
provider "azurerm" {  
  features {}  
}
```

OPERATOR**MEANING****=**

Exactly this version

!=

Any version except this

>= <=

Comparisons

~> 4.0

Allow 4.x, not 5.0

~> 1.0.4

Allow 1.0.5+, not 1.1

**~>**

is the safe default — patch/minor updates,
no surprise majors.

02

Resources

The building blocks — and the graph Terraform builds between them.

A resource, dissected

HCL

```
resource "azurerm_resource_group" "app" {  
  name      = "app-rg"  
  location = "westeurope"  
}
```

🔗 Reference pattern:

`type.name.attribute` →

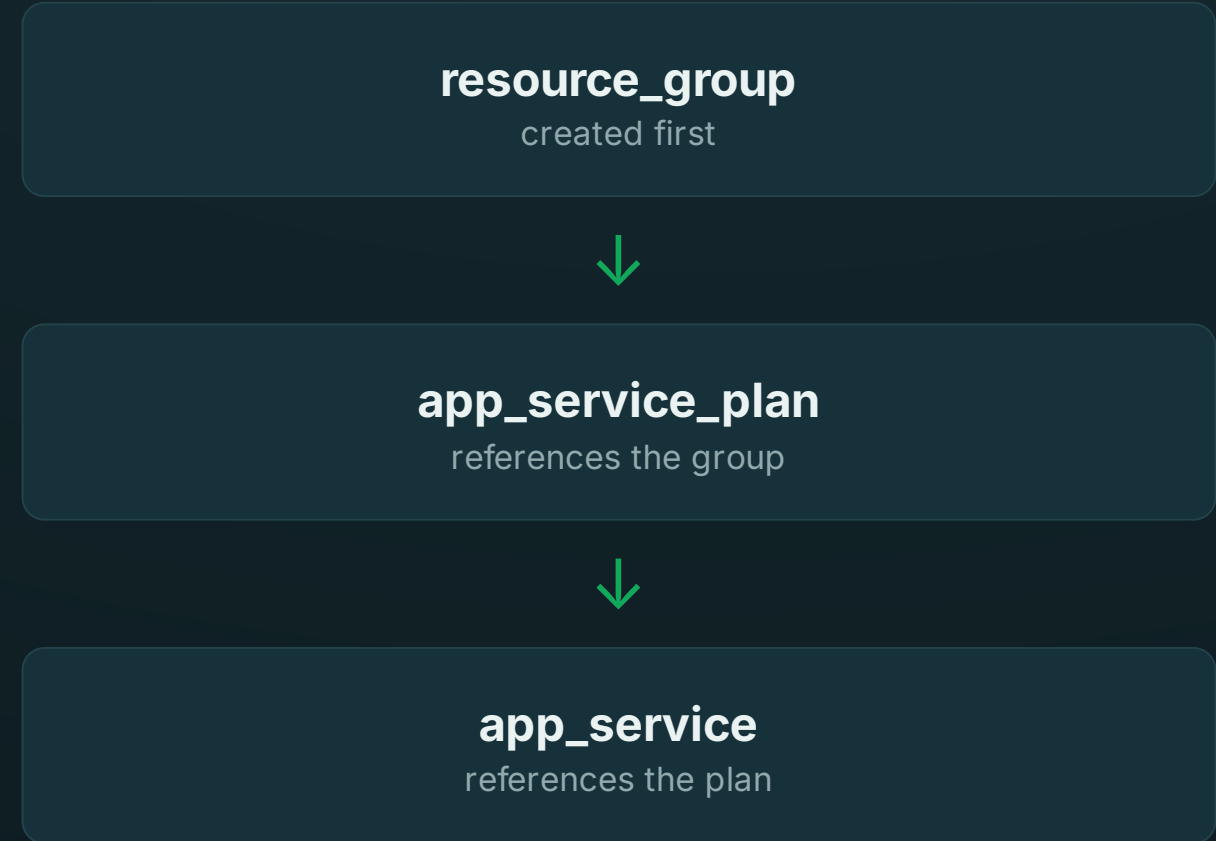
`azurerm_resource_group.app.id`

- **resource** — the top-level keyword
- **type** — `azurerm_resource_group`, defined by the provider
- **name** — a local label you choose (never a variable)
- **arguments** — the properties for that resource

Dependency mapping

Terraform creates resources **in parallel** by default — then builds a dependency graph from the references in your code.

- Reference one resource from another → implicit ordering
- The graph decides the safe creation sequence
- Circular references are detected and rejected



03

Terraform State

Terraform's memory of what it has built.

What is Terraform state?



A snapshot

The state of your infrastructure at a point in time, in `terraform.tfstate`.



The map

Which resources exist, their IDs & attributes, and how they relate.



The diff engine

Every plan compares **code** ↔ **state** ↔ **reality** to decide what to change.

⚠ State is a JSON file that only Terraform should edit. **Never** hand-edit it.

How state works

01

Initial apply

Create resources, write them to state



02

Later runs

Compare state vs. updated code



03

Track changes

Apply only the necessary diffs



`terraform plan` prints exactly this comparison as a list of create / change / destroy actions.

Organize your code

Terraform merges all `.tf` files in a folder — but the community follows one layout.

main.tf

Resources / module calls that build the infra.

variables.tf

Input variable declarations with types & defaults.

outputs.tf

Values returned after apply.

versions.tf

Terraform & provider version constraints.



`.terraform.lock.hcl` pins exact provider versions & hashes — commit it like source code.

 LIVE DEMO

Organize code & drive the CLI

Split a project into files, run **fmt** · **validate** · **plan** · **apply**, use plan files, `-auto-approve`, and `destroy`.

Demo · 02 Provider · GitHub

Organize & structure your project

Refactor last session's repo project into `main / variables / outputs / versions`, format it, and run the plan-file workflow.

You'll practise: file layout · `fmt` · `validate` · saved plan files · reading state

Key takeaways

✓ Providers = the bridge

One HCL language reaching every platform via plugins.

✓ Resources form a graph

References create implicit, safe ordering.

✓ State is the source of truth

Every plan diffs code ↔ state ↔ reality.

✓ Next up

Session 03 — variables, locals & outputs.



Questions?

Up next — [Session 03: Variables & Outputs.](#)